
Spring MVC

-- Spring MVC is a web application framework that follows the Model-View-Controller (MVC) design pattern.

-- The MVC pattern separates the application into three main components : model, view, and controller.

-- Its is a lightweight and flexible framework that provides a wide range of features including :

1 - Request Handling (Processes browser requests through the controller and returns the response)

2 - View Resolution (Maps the returned view name to the correct template (JSP | home.jsp)

3 - Data binding and validation (Automatically binds form data to objects and validates it)

4 - Exception handling (Handles errors properly to prevent the application from crashing | like try or catch)

Spring MVC Advantages

- Easy to use
- It can be customized to meet the specific needs of our application.
- It can be used to build small to large-scale web applications.
- It is well-supported and has a large community of users.

Key Features of Spring MVC

1 - DispatcherServlet

-- Every request first comes to DispatcherServlet
-- Then it decides which Controller should handle it
-- After that it sends the request to that controller and returns the response back to the user.

2 - Controllers

-- Its is the responsible for handling user requests and updating the model.

3 - Views

-- They take the data (Model) sent by the Controller and display it to the user as output.
-- Example: JSP, Thymeleaf, HTML pages

4 - Handler Mappings

-- When a request comes like : /login or /signup or etc
-- Handler Mapping finds : which controller/method should run for it.

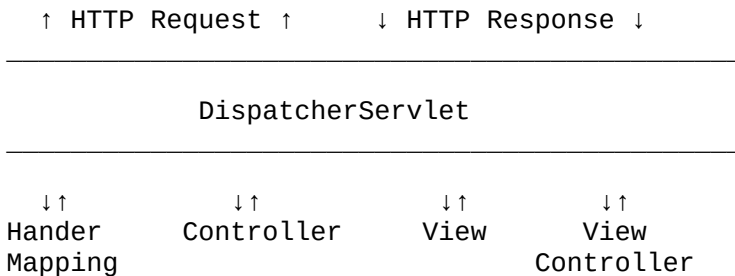
5 - View Resolvers

```
-- Controller returns a view name like : "home"
-- View Resolver converts it into an actual file like : /WEB-INF/views/home.jsp
(Then that page displays the model data.)
```

6 - Locale Resolvers

```
-- They decide the language/region settings for the user.
-- Example : English / Hindi based on browser or user choice
```

```
-----
--
--          >>      SPRING MVC WORKING STRUCTURE
--
-----
```



Spring MVC Architecture

```
-- The MVC pattern separates the application into three main components : model,
view, and controller.

-- Model : The model represents the data of the application.
-- View : The view is responsible for rendering the model data to the user.
-- Controller : The controller is responsible for handling user requests and
updating the model.
```

How does Spring MVC work?

```
-- The request response cycle
-- DispatcherServlet
-- URL Mapping
-- Controller
-- Model
-- View
-- Response
```

Spring MVC Annotations

1 - @Controller

```
-- This class will handle web requests.
-- It receives HTTP requests like /hello
-- Runs the correct method
-- Returns a response (page or message)
```

2 - @RestController

- @RestController is used for REST APIs.
- It is like @Controller
- But it directly returns data (JSON/XML) instead of returning a web page
- Best for backend API development

3 - @RequestMapping

- Connects a URL to a controller class/method
- Also can define GET / POST / PUT / DELETE
- Example : /login should call this method.

4 - @RequestParam

- Takes a value from the URL query parameter or form input
- Example: /search?name=Anmol → "Anmol" is captured.

5 - @RequestBody

- Takes data from the request body (mostly JSON)
- Converts it into a Java object used in POST/PUT APIs.

6 - @ResponseBody

- Sends the method return value directly as response data.
- Mostly returns JSON/XML (No view page like JSP)

7 - @ResponseStatus

- Sets the HTTP status code for the response
- Example: 200 OK, 201 Created, 404 Not Found etc.

--

1 - InternalResourceViewResolver (Configuration | Meta Data)

- It is used to convert view name into actual JSP file path
- Mostly used in Spring MVC + JSP

Example:

- Controller returns: "home"
- It becomes: /WEB-INF/views/home.jsp

2 - @GetMapping, @PostMapping

- These are shortcuts of @RequestMapping
- Used to define HTTP method clearly

@GetMapping → handles GET request

@PostMapping → handles POST request

Example:

/login open page → GET

/login submit form → POST

3 - @PathVariable

-- Takes value directly from the URL path
-- Used when URL contains dynamic data

Example:

/user/101 → 101 is captured as id

4 - Model, ModelMap, ModelAndView

-- These are used to send data from Controller to View

Model

-- Simple way to add data

Example: model.addAttribute("name", "Anmol")

ModelMap

-- Same as Model but works like a Map

Example: modelMap.put("name", "Anmol")

ModelAndView

-- Contains both Model + View name together

Example: view = "home" and data = "name"

5 - Form Validation (@Valid, BindingResult)

@Valid

-- Checks validation rules on a Java object

Example: name should not be empty email should be valid

BindingResult

-- Stores validation errors

-- Used to check if form has mistakes

Example:

If errors exist → return same form page again

6 - Exception Handling (@ExceptionHandler, @ControllerAdvice)

@ExceptionHandler

-- Handles exceptions inside a Controller

Example: handle NullPointerException, UserNotFoundException

@ControllerAdvice

-- Global exception handler for all controllers

-- Used to handle errors in one common place

Example:

If any controller throws error → show common error page / JSON response

--

These components, annotations, and configurations are essential in Spring MVC for request handling and response generation.